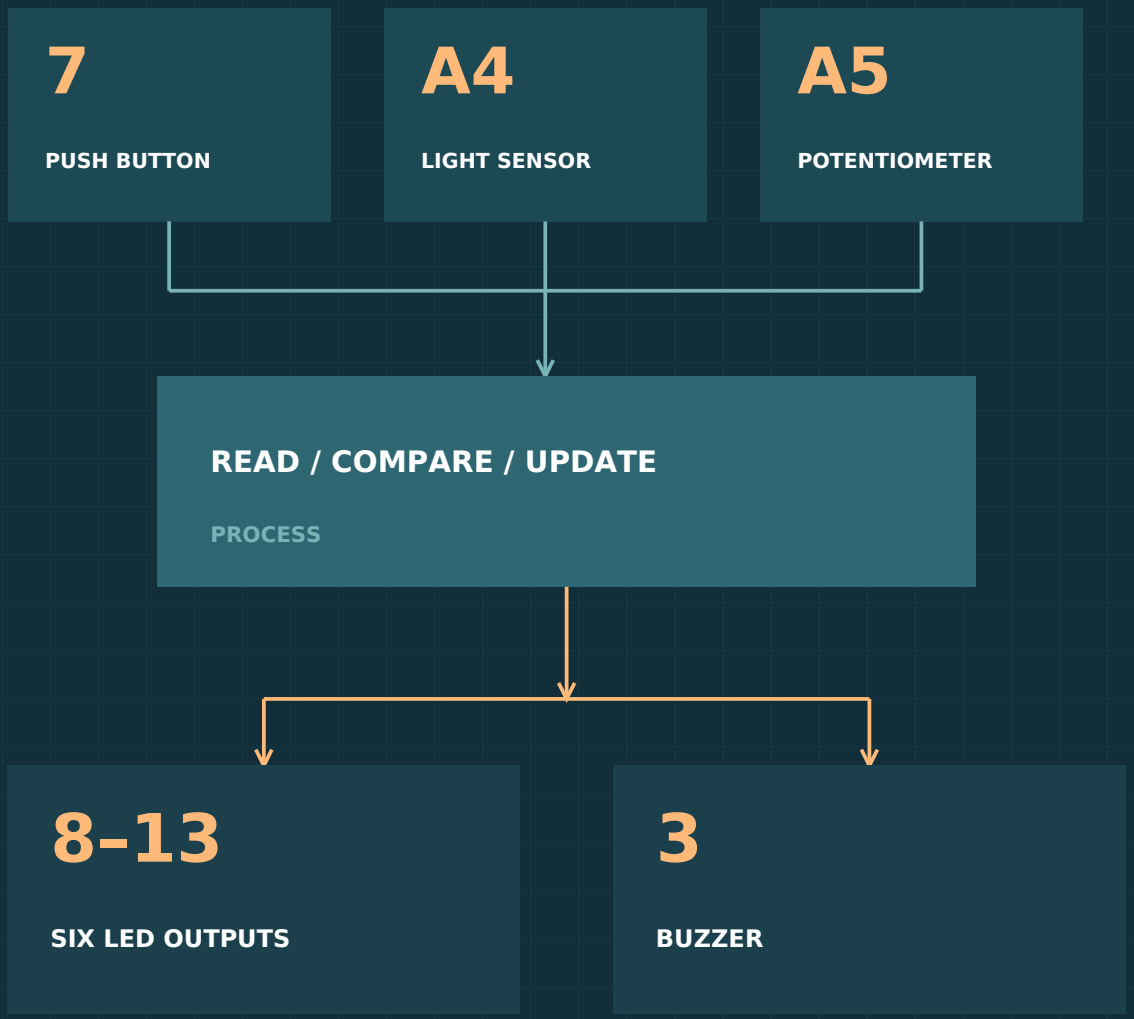
YEAR 8 / DIGITAL TECHNOLOGIES

Crack the Code

STUDENT WORKBOOK

THINKERSHIELD

A signal becomes a decision



Conceptual signal flow · fixed course pin map

Design a system that senses, decides and responds.

Learn the logic, trace the code and use evidence to explain an alarm built with the fixed ThinkerShield inputs and outputs.

NAME _____ **CLASS** _____

Conceptual signal portrait using the course pin map. This is not a physical board or wiring diagram.

50 A4 pages including cover and contents · Duplex long edge · Teacher guide separate

READ • TRACE • TEST • EXPLAIN

Your learning route

Use the book for your evidence and the website for further practice.

Module	Learning	Pages	Studio
1	Systems that sense and respond	3–6	6
2	Make light obey the code	7–10	10
3	Plan the logic before coding	11–15	14–15
4	Read changing data	16–20	19–20
5	Sense the light	21–25	24–25
6	Make the code choose	26–30	29–30
7	Turn numbers into sound	31–34	34
8	Combine parts into systems	35–39	38–39
9	Design the final alarm	40–45	43–45
10	Debug, improve and explain	46–50	49–50

Course website

https://stevencowell.github.io/Year-8-Crack-the-Code/

Each teaching page names the matching learning activity and identifies which original question appears in print. The remaining questions and complete additional responses stay at that exact section online. The Busy Work bank provides optional extension; your teacher selects what to use.

One evidence record

The online folio groups existing work into three stages. Understand and plan the system covers modules 1 to 3; Read inputs and make decisions covers modules 4 to 7; Combine, produce and evaluate covers modules 8 to 10. These stages re-display existing responses and studios. Keep one connected record.

Current classroom directions

This workbook supports the course; it does not set formal dates, marks or submission rules. Your teacher confirms equipment access, safe practical routines, file naming, collaboration and hand-in arrangements. Archived 2025 task attachments were not used as current instructions.

Save and back up

Website practice is saved in the current browser. It does not upload to Drive or turn in work to Classroom. Keep the tested code version and a backup as directed; verify the actual submission route before hand-in.

1.1 INPUT → PROCESS → OUTPUT

Follow the information

A control system joins sensing, a decision and a response.

A control system receives information through an input, makes a decision during processing, then causes an output. A torch button, automatic door and school bell use different components, but the same input–process–output pattern helps explain each system.



Figure 1. A night-light signal path. The rule must match the observed sensor behaviour; the diagram does not prescribe a threshold.

For an alarm, a sensor supplies data, the program compares that data with a rule, and LEDs or a buzzer communicate the response. The three stages are connected: changing the input or processing rule can change the output.

A part list cannot show why the alarm responds. Follow one reading through the sketch: what is measured, what comparison is made, and which output changes? The LDR supplies a value. The program gives that value meaning by comparing it with a rule. The LED communicates the decision.

1. Which action best applies the theory?

- ○ A list components without explaining the information flow
- ○ B change several unrelated parts and hope
- ○ C name the real input, the decision made by the code and the resulting output

Explain one everyday control system using input, processing and output.

Choose a real system, name the three stages and show how information moves between them.

Continue online: Learning activity 1.1, questions 1–2 and 4–10. Printed check = website question 3.

Open the matching course section

1.2 MICROCONTROLLERS MAKE DECISIONS

Microcontrollers make decisions

Read any control system as input, processing and output.

A microcontroller is a small programmable computer designed to control a system. The Arduino runs one uploaded sketch repeatedly and uses its pins to communicate with connected inputs and outputs.

The ThinkerShield places classroom components around the Arduino so the focus can stay on code and systems thinking. The board does not invent the behaviour: the student's algorithm and code tell the microcontroller what data to read and what action to take.

One program gives parts a job

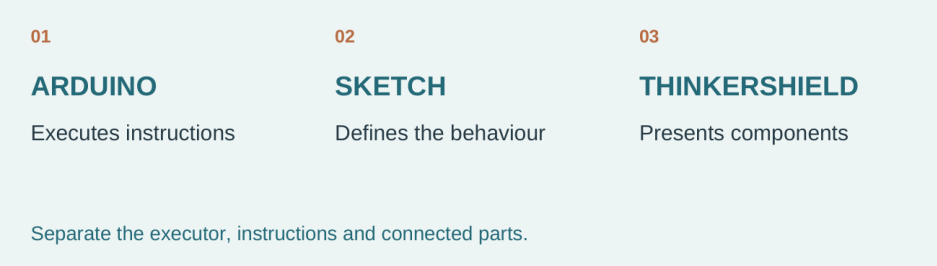


Figure 1.2. The board executes the sketch; the shield supplies components. These are roles, not a physical assembly drawing.

The same shield can be used for a night light or a button-triggered alert because the uploaded sketch changes the behaviour. The Arduino executes those instructions; the ThinkerShield presents the components the program can read or control. If the wrong LED responds, first compare the code's destination with the fixed map. Replacing the shield without checking the sketch does not explain the fault.

2. Which correction best addresses the misconception "the ThinkerShield is the program"?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C the shield provides components while the uploaded sketch provides behaviour

Name the part that supplies components and the part that supplies behaviour.

Continue online: Learning activity 1.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

1.3 WORK WITHIN THE REAL SYSTEM

Work within the real system

Read any control system as input, processing and output.

This course uses fixed ThinkerShield connections: push button on digital pin 7, LDR on analogue input A4, potentiometer on analogue input A5, six LEDs on pins 8–13 and the buzzer on pin 3. Pin numbers in code must match the actual board.

Handle the equipment only as directed, connect the blue USB cable carefully and keep the board on a clear, stable workspace. Lesson access, damaged equipment procedures and any additional classroom controls remain Teacher to confirm.

A pin is an address

INPUTS	BUTTON	7
	LDR	A4
	POTENTIOMETER	A5
OUTPUTS	SIX LEDs	8–13
	BUZZER	3

Figure 1.3. Fixed course connections. This reference identifies addresses; it is not a wiring procedure.

The source scenario sends a buzzer command to pin 11. That number belongs to the LED range, while the authorised buzzer is on pin 3. The meaningful correction is to compare the intended output with the fixed map, change the mistaken address and retest under teacher direction. Random pin changes hide the reason for any improvement. The map is an information check, not permission to rewire the board.

3. A sketch sends the buzzer command to pin 11. What should happen next?

- ○ A change the sketch to the authorised buzzer pin 3 and retest under teacher direction
- ○ B change pin numbers randomly when the board does not respond
- ○ C hide the result and remove it from the record

Mark an input and an output in the fixed map. Explain how you distinguished them.

Continue online: Learning activity 1.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

PREPARE FOR THE HARDWARE AND IPO WORK.

Control-system detective

Identify one complete signal path, then connect the same thinking to the fixed ThinkerShield hardware.

1. Study the IPO and pin-map diagrams before writing.
2. Choose one everyday control system and name the real input, rule and output.
3. Use the fixed pin map to plan the final alarm and state one check you will make before connecting or uploading.

Choose an everyday control system.

What information enters it?

What rule or decision is applied?

What changes as a result?

Map the final alarm inputs and outputs to their fixed pins.

Component	Fixed pin(s)
Button	
LDR	
Potentiometer	
LEDs	
Buzzer	

What will you check before connecting or uploading?

Online extension: Trace the signal path (busy-work.html#busy-01); Know the board (busy-work.html#busy-02).

Open the matching course section

2.1 A SKETCH HAS A WORKING STRUCTURE

Read the sketch in order

Configuration happens once; behaviour inside loop repeats.

An Arduino sketch normally contains setup() and loop(). Setup runs once after the board starts or resets. It prepares the system, including setting an LED pin as an output. Loop then repeats the behaviour inside it. Braces group instructions; semicolons finish most commands.

The section example holds the LED on

```
void setup() { pinMode(8, OUTPUT); }
void loop()  { digitalWrite(8, HIGH); }
```

There is no LOW command here. Repeating the on command does not, by itself, make a flash.

The original studio Blink creates a complete cycle

```
void setup() {
  pinMode(8, OUTPUT);
}
void loop() {
  digitalWrite(8, HIGH);
  delay(500);
  digitalWrite(8, LOW);
  delay(500);
}
```

Figure 2. Complete code transcribed from the source Blink diagram, including the closing brace. Pin 8 is one of the fixed ThinkerShield LED pins.

Work through one cycle: HIGH → wait 500 ms → LOW → wait 500 ms. The loop then returns to its first command. Both waits matter: one makes the on state visible and one makes the off state visible.

Annotate a short Blink sketch to explain setup, loop, braces and semicolons.

Draw leaders to the code and add notes on the rules above. Trace the code before any teacher-directed upload. Module 2 checks are on page 9; the whole experiment record is on page 10.

Continue online: Learning activity 2.1, questions 1–2 and 4–10. Printed check on page 9 = website question 3.

Open the matching course section

2.2 DIGITAL OUTPUTS HAVE TWO STATES

Digital outputs have two states

Build reliable digital outputs with setup, loop and timing.

digitalWrite(pin, HIGH) sends the on state to a configured digital output; LOW sends the off state. On the ThinkerShield, the LED pin must match the LED being controlled.

Changing a pin number changes the destination, not the meaning of HIGH or LOW. Predict the target and state before uploading so the result can be checked against an expectation.

Destination and state are different

```
digitalWrite(8, HIGH);
digitalWrite(9, LOW);
```

Illustrative trace using authorised pins: after these two commands, LED 8 is on and LED 9 is off. The sequence assumes both pins have been configured as outputs.

In digitalWrite(8, HIGH), 8 selects an LED output and HIGH selects its on state. A later command to pin 9 addresses another LED; it does not automatically turn pin 8 off. Trace each command and keep a record of each destination's latest state. This makes a prediction testable: "LED 8 is on and LED 9 is off" is clearer than "the lights work".

For a teacher-supplied two-LED sequence, record the final state of each addressed LED.

Module 2 original checks are together on page 9. Continue online: Learning activity 2.2, questions 1–8 and 10, plus the complete guided response. Printed check = website question 9.

Open the matching course section

2.3 TIMING MAKES A SEQUENCE VISIBLE

Compare complete cycles

Circle one answer for each original course question.

delay(milliseconds) pauses the program. One thousand milliseconds equals one second. A flash needs on, wait, off and wait. Shorter total waits make repeated cycles faster; delay also pauses the whole sketch, which can affect a combined system.



Figure 3. Source timing examples: A totals 1000 ms; B totals 500 ms. Compare the complete ON + OFF cycle.

4. Which action best applies the theory?

- ○ A remove braces or scatter commands outside functions
- ○ B change several unrelated parts and hope
- ○ C configure the pin in setup and place repeated output behaviour in loop

5. Which correction best addresses the misconception “HIGH means a physically higher LED”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C HIGH is a logic state sent by the microcontroller

6. Both 1000 millisecond delays are changed to 200 milliseconds. What should happen next?

- ○ A expect the LED to flash more quickly and confirm by observing repeated cycles
- ○ B change several timing values without predicting the effect
- ○ C hide the result and remove it from the record

Use your predictions in Getting Flashy laboratory on page 10. Keep the observed result separate from the result you expected.

Continue online: 2.1 questions 1–2, 4–10; 2.2 questions 1–8, 10; 2.3 questions 1–9. These printed checks are website questions 3, 9 and 10 respectively.

Open the matching course section

SOURCE PATHWAY 1–2: GETTING FLASHY AND EVEN FLASHIER.

Getting Flashy laboratory

Predict, build and explain a visible LED sequence using the correct sketch structure and deliberate timing.

1. Read the annotated sketch and compare the two timing diagrams.
2. Predict the first LED behaviour before uploading.
3. Under teacher direction, test the supplied task, change one delay only and compare the new cycle with your prediction.
4. Record the observed sequence and any verification or upload fault you corrected.

Predict the first LED behaviour before upload.

Record the actual result.

Change one delay value. What changed?

Plan a two-LED sequence with exact timing.

Step	LED pin and state	Time or delay
1		
2		
3		

Record one verification or upload fault and its correction.

What evidence proves the sequence repeated correctly?

Online extension: Put the sketch in order (busy-work.html#busy-03); Engineer a visible signal (busy-work.html#busy-04).

Open the matching course section

3.1 ALGORITHMS ARE ORDERED SOLUTIONS

Algorithms are ordered solutions

Turn ideas into precise algorithms, pseudocode and flowcharts.

An algorithm is a finite sequence of unambiguous steps used to solve a problem. For a flashing LED, the order matters: configure the output, turn it on, wait, turn it off, wait and repeat.

A useful algorithm is precise enough for another person to follow and test. “Make it flash nicely” is an intention, not an algorithm.

Dry-run the given value

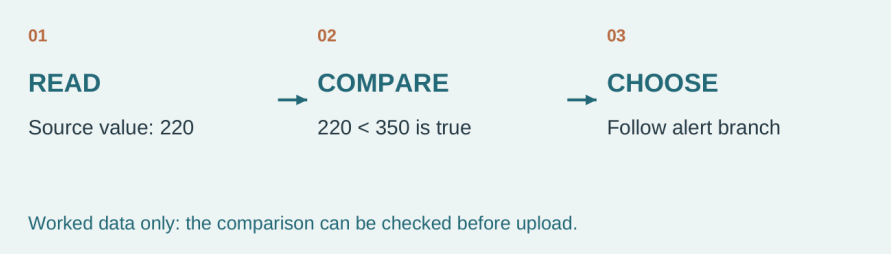


Figure 3.1. Source worked value 220 follows the true path for value < 350. These numbers are illustrative.

The source example reads 220 and compares it with “below 350”. Because 220 is less than 350, the alert branch is selected. No board is needed to test that logical step. To test the whole algorithm, follow its reading, comparison and output stages in order, then trace a value on the other side. These are worked numbers, not a threshold authorised for your measured sensor.

7. Which action best applies the theory?

- ○ A use vague instructions that require the reader to guess
- ○ B change several unrelated parts and hope
- ○ C write each operation in the order it must occur

Explain why “make it flash nicely” cannot be dry-run reliably.

Continue online: Learning activity 3.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

3.2 PSEUDOCODE KEEPS ATTENTION ON LOGIC

Pseudocode keeps attention on logic

Turn ideas into precise algorithms, pseudocode and flowcharts.

Pseudocode uses structured plain language to express an algorithm without demanding exact programming syntax. Words such as START, READ, IF, ELSE, SET and REPEAT make the control flow visible.

Indentation shows which steps belong inside a branch or loop. Consistent names, such as lightReading, reduce confusion when the plan becomes code.

Keep the plan readable

```
REPEAT
  READ lightReading
  IF the measured condition meets the alert rule
    SET all relevant alert outputs
  ELSE
    SET all relevant normal outputs
```

Figure 3.2. One complete conditional plan uses consistent names and indented alternative actions.

Indenting the normal and alert actions under their own branches makes their relationship visible. A reader should be able to answer: when does this action happen, and what happens instead if the condition is false? Use one name for each value throughout. The plan is useful before syntax is correct, but it must still identify a complete decision and a return to the next reading.

8. Which correction best addresses the misconception “pseudocode has one compulsory programming grammar”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C it should be consistent and unambiguous, but it is not a specific programming language

Underline the decision and mark the two paths in your studio pseudocode.

Continue online: Learning activity 3.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

3.3 FLOWCHARTS SHOW PATHS AND DECISIONS

Flowcharts show paths and decisions

Turn ideas into precise algorithms, pseudocode and flowcharts.

A flowchart uses agreed shapes: an oval for start or end, a rectangle for a process, a parallelogram for input or output, and a diamond for a decision. Arrows show the direction of control.

A decision must be a question or condition with labelled paths such as Yes/No or True/False. In the new syllabus, branching means selecting different actions depending on a condition.

A diamond creates a choice

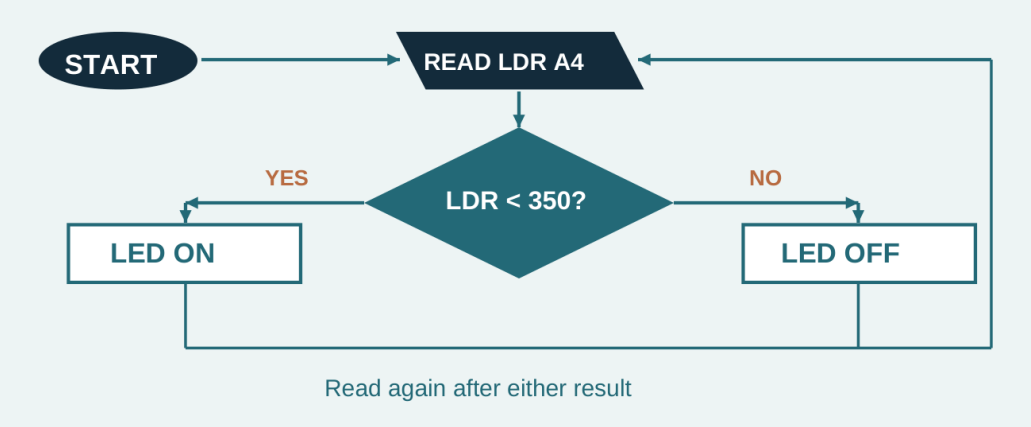


Figure 3.3. Source example redrawn with both labelled paths and a return to the next reading.

Follow the source example’s decision “LDR < 350?” in both directions. Label the true route and the false route so another reader can trace the result. The arrow returning to the next reading matters: without it, the picture may describe a one-off action rather than a responsive system. Match symbols to their jobs, and compare the finished chart with the words in your algorithm.

9. The question “Is the button pressed?” appears in a process rectangle. What should happen next?

- ○ A move the condition to a decision diamond and label both paths
- ○ B draw one unlabelled path from a decision
- ○ C hide the result and remove it from the record

Describe what a missing No or False path prevents a reader from checking.

Continue online: Learning activity 3.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

PLAN CODE BEFORE TYPING IT.

Algorithm and flowchart studio 1 of 2

Represent one alarm rule as an ordered algorithm, pseudocode and a two-path decision.

1. Use the dry-run diagram to follow a sample value through the rule.
2. Read the flowchart from START and follow both the YES and NO arrows.
3. Write your own repeating alert logic in three matching forms.
4. Dry-run one value for each path and correct any missing or ambiguous step.

Write the ordered steps for a repeating alert.

Use the same rule in both representations.

Rewrite the same logic as pseudocode.

Use the same rule in both representations.

Open the matching course section

PLAN CODE BEFORE TYPING IT.

Algorithm and flowchart studio 2 of 2

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Draw or describe a complete flowchart for one alarm rule.

State the exact decision question.

Draw the complete flowchart here. Label the decision and both paths; include a return to reading.

What happens when the condition is true?

What happens when it is false?

Dry-run both paths and record any correction.

Test value / condition	Chosen path and output	Correction

Online extension: Algorithm before upload (busy-work.html#busy-05); Follow the branch (busy-work.html#busy-06).

Open the matching course section

4.1 ANALOGUE READINGS SHOW A RANGE

Analogue readings show a range

Measure analogue inputs and turn ranges into useful behaviour.

An analogue input represents many possible values rather than only two states. On this equipment, analogRead() returns a value across the Arduino input range. The potentiometer on A5 changes as its knob turns.

Record actual low, middle and high readings from the board instead of assuming every component reaches identical extremes.

Read the pattern without copying it

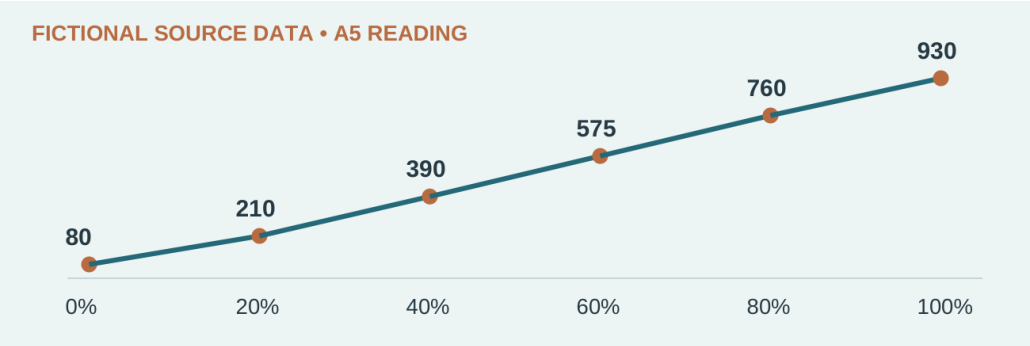


Figure 4.1. Fictional source A5 readings at six knob positions. Collect your own board readings in the studio.

In the source's fictional dataset, moving from 0% to 100% changes the reported value from 80 to 930. Intermediate values increase too, but this example does not prove that your board has those endpoints. A useful record names the knob condition as well as the number. Keeping the two together lets someone compare the measurement with the physical condition that produced it.

10. Which action best applies the theory?

- ○ A assume the only possible readings are zero and one
- ○ B change several unrelated parts and hope
- ○ C read A5 repeatedly and observe values in the Serial Monitor

Using the given sample, state the lowest and highest value and describe the trend.

Continue online: Learning activity 4.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

4.2 MAPPING CONVERTS ONE RANGE TO ANOTHER

Give a reading a useful scale

A calculation changes the number, not the component.

map() converts a value from one numeric range into another. A potentiometer reading can become a useful delay, number of lit LEDs or permitted output range. First measure the real A5 input range; do not assume that every component reaches identical extremes.

```
level = map(potReading, lowInput, highInput, 0, 6);
```

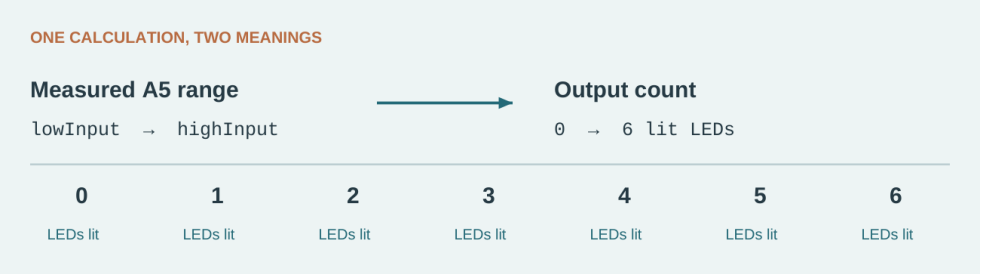


Figure 4. The original section code maps to a count from no LEDs to six LEDs. It does not itself switch the LEDs on.

Read the arguments in order

potReading is the current input. lowInput and highInput are calibration endpoints. Here the low endpoint becomes 0 and the high endpoint becomes 6. A separate decision or loop uses the result to control outputs.

Keep count and index separate. Six items have indexes 0–5; a count of lit LEDs can be 0–6. The activity diagram uses a separate 0–5 level scale. Define the number’s meaning; the fixed pins remain 8–13.

11. Which correction best addresses the misconception “map changes the electrical component itself”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C map performs a calculation on the numeric reading

Underline the input range and circle the output range in the code. Explain what level means.

Inputs outside the calibrated range may need constraining. Use your own measured data.

Continue online: Learning activity 4.2, questions 1–8 and 10, plus the complete guided mapping response. Printed check = website question 9.

Open the matching course section

4.3 CALIBRATE BEFORE JUDGING BEHAVIOUR

Calibrate before judging behaviour

Measure analogue inputs and turn ranges into useful behaviour.

Calibration connects raw readings to the real conditions being measured. Take several readings at useful positions, note variation and choose thresholds or map endpoints supported by evidence.

Change one variable at a time during testing. If both code and hardware position change together, it becomes difficult to identify the cause of a result.

Separate measurement from judgement

Condition held steady	One factor changed	What to compare
Same input condition	One code choice	Expected and observed output
Same code	One named input condition	Repeated input readings

Suppose the displayed result surprises you. First ask whether the input readings support the endpoints or boundary in the code. Take repeated measurements at clearly named conditions and compare the pattern. If you turn the knob and change the mapping at the same time, you cannot tell which caused the difference. Keep one condition steady while changing one chosen factor, then repeat the same check.

12. Readings near the chosen threshold move above and below it rapidly. What should happen next?

- ○ A collect more data and adjust the rule or tolerance based on evidence
- ○ B copy a threshold without testing the actual device
- ○ C hide the result and remove it from the record

State one thing you would keep the same during a calibration retest.

Continue online: Learning activity 4.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

SOURCE PATHWAY 3–5: POT BASICS, A LIGHT DIMMER AND POT LED BOUNCER.

POT investigation laboratory 1 of 2

Collect changing A5 data, describe its pattern and map the measured range to a useful output.

1. Read the sample A5 graph and mapping diagram before collecting data.
2. Under teacher direction, measure low, middle and high potentiometer positions.
3. Compare your range with the sample pattern; do not copy the sample as your measurement.
4. Predict three mapped outputs, test them and explain how calibration changed the code choice.

Record and interpret a low, middle and high potentiometer reading.

Use the matching studio fields across these two pages to record and interpret your readings. One record serves both purposes.

Lowest observed A5 reading and knob condition.

Knob condition	Reading(s)

Middle observed A5 reading and knob condition.

Knob condition	Reading(s)

Highest observed A5 reading and knob condition.

Knob condition	Reading(s)

State the measured input and intended output ranges.

Open the matching course section

SOURCE PATHWAY 3–5: POT BASICS, A LIGHT DIMMER AND POT LED BOUNCER.

POT investigation laboratory 2 of 2

Collect changing A5 data, describe its pattern and map the measured range to a useful output.

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Predict three mapped outputs.

Input	Predicted output	Observed output

Explain how the LED Bouncer uses changing data.

What did calibration change in your code choice?

Online extension: Read analogue evidence (busy-work.html#busy-07); Map data to output (busy-work.html#busy-08).

Open the matching course section

5.1 THE LDR TURNS LIGHT INTO DATA

The LDR turns light into data

Use the LDR to detect environmental change and trigger outputs.

The light-dependent resistor on A4 contributes to a circuit whose analogue reading changes with light conditions. The exact direction and size of the change should be observed on the supplied board.

The sensor provides data; it does not decide what counts as dark. The program's threshold and branch create that meaning.

The sensor does not define dark

Fictional source condition	A4 range
Bright room	720–850
Dim room	390–520
Covered sensor	80–210

Source worked-data example: threshold 300 separates the covered range from the other two ranges. These are not your measurements. The next section uses a separate threshold-400 boundary example.

A covered sensor produces a reading, but the word "dark" is a category supplied by a rule. Before choosing that rule, record several A4 values in bright, normal and covered conditions. If readings vary, one value may be misleading. Compare the groups and observe whether they rise or fall as light changes. Your comparison operator must follow that evidence, not a remembered example from a different circuit.

13. Which action best applies the theory?

- ○ A state that a higher number always means darker without checking the board
- ○ B change several unrelated parts and hope
- ○ C read A4 under named lighting conditions and record the actual direction of change

Why should the condition and repeated values be recorded together?

Continue online: Learning activity 5.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

5.2 THRESHOLDS CREATE CATEGORIES

Thresholds create categories

Use the LDR to detect environmental change and trigger outputs.

A threshold is a chosen boundary used to separate readings into categories. A night light might treat one side as “dark” and the other as “bright”.

A useful threshold sits between measured condition ranges and is tested near the boundary. The comparison operator—such as < or >—must match the observed direction of the sensor data.

Test the boundary as well as the categories

Source example reading	Test: reading < 400	Path
120	True	Alert
399	True	Alert
400	False	Normal
401	False	Normal
780	False	Normal

For the source’s illustrative rule LDR < 400, a reading of 399 selects the true path, while 400 does not. Equality belongs to the false path because the operator is strictly less than. This is a paper trace, not a recommended threshold. A real threshold must be justified using your measured ranges. Include a value immediately below, at and above your chosen boundary in the test plan.

14. Which correction best addresses the misconception “a threshold is the same as the sensor reading”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C the reading changes while the threshold is a chosen comparison value

For the illustrative rule LDR < 400, which path does 401 follow, and why?

Continue online: Learning activity 5.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

5.3 A NIGHT LIGHT IS A FEEDBACK RULE

A night light is a feedback rule

Use the LDR to detect environmental change and trigger outputs.

A basic night-light program repeatedly reads the LDR, compares the value with a threshold, and updates an LED. Repetition matters because the environment can change after the first reading.

Testing should include both expected conditions, a value near the threshold and recovery when light returns. A system that turns on but never returns to normal is incomplete.

Recovery completes the rule

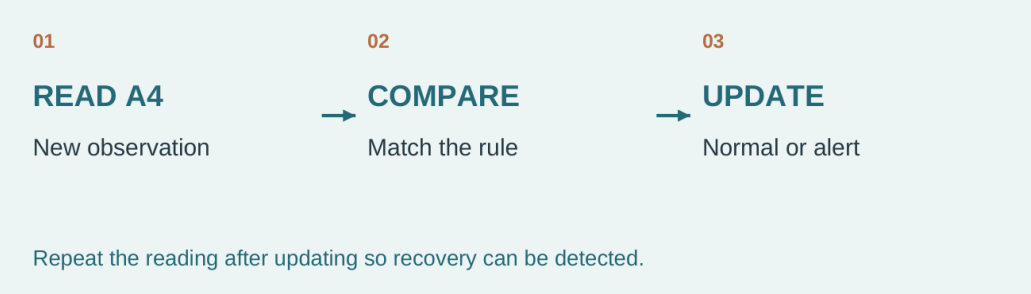


Figure 5.3. A response loop reads again after updating; the next reading can lead to recovery.

A night light can respond correctly once and still fail as a system. If it never reads again, it cannot notice that the room has become brighter. If the false branch never updates the LED, an old alert state may remain. Trace a complete trip from normal conditions into the trigger condition and back to normal. The return result is evidence of recovery, not simply another example of switching on.

15. The LED turns on in darkness but stays on when light returns. What should happen next?

- ○ A check the else branch and ensure it writes the LED off state
- ○ B read the sensor only once in setup
- ○ C hide the result and remove it from the record

Name the observation that would show a successful recovery test.

Continue online: Learning activity 5.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

SOURCE PATHWAY 6–7: LDR BASICS AND LDR NIGHT LIGHT.

LDR night-light investigation 1 of 2

Use repeated A4 evidence to choose, test and justify a light threshold.

1. Use the worked threshold and calibration examples on pages 21–22 to distinguish measured ranges from the decision boundary.
2. Collect repeated bright, normal and covered readings under teacher direction.
3. Describe whether A4 rises or falls as light decreases.
4. Choose a threshold from your evidence, then test immediately below, at and above it plus a recovery condition.

Record repeated A4 readings in bright conditions.

Condition detail	Repeated readings

Record repeated A4 readings in normal conditions.

Condition detail	Repeated readings

Record repeated A4 readings when covered.

Condition detail	Repeated readings

Does the reading rise or fall as light decreases?

Open the matching course section

SOURCE PATHWAY 6–7: LDR BASICS AND LDR NIGHT LIGHT.

LDR night-light investigation 2 of 2

Use repeated A4 evidence to choose, test and justify a light threshold.

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Choose and justify an LDR threshold from a small set of readings.

Use the matching studio fields below for this complete guided response. One record serves both purposes.

Choose and justify a threshold.

Record the alert test result.

Condition	Expected output	Actual output
Below boundary		
At boundary		
Above boundary		

Record what happened when light returned.

Online extension: Set a light threshold (busy-work.html#busy-09); Calibrate the night light (busy-work.html#busy-10).

Open the matching course section

6.1 READ A DIGITAL INPUT

Read a digital input

Read the button, branch with if/else and remember state.

digitalRead(7) reads the ThinkerShield push button. The supplied button arrangement may report LOW when pressed and HIGH when released, so observe the real values before writing the condition.

Name the stored value clearly, such as buttonState. A meaningful name makes later comparisons easier to trace.

Observe before you compare

Button condition	Observe before coding
Released	Record the actual pin 7 value
Pressed	Record the actual pin 7 value

Source worked rule only: if the observed pressed state is HIGH, the table below follows. It does not establish the real state of your board.

Reading	Condition	Branch	LED
HIGH	True	if	ON
LOW	False	else	OFF

A button reading is an electrical logic value, not the word “pressed”. The course example warns that pressed may read LOW. Measure released and pressed on the supplied arrangement and keep both in your record. Only then can a condition such as buttonState == LOW be interpreted as a pressed test. A video’s active-HIGH example cannot establish the state of this particular board.

16. Which action best applies the theory?

- ○ A assume pressed must always equal HIGH
- ○ B change several unrelated parts and hope
- ○ C read pin 7 and verify which value represents pressed on the actual board

Why is a button condition unreliable if only the pressed value was guessed?

Continue online: Learning activity 6.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

6.2 IF/ELSE CREATES TWO PATHS

If/else creates two paths

Read the button, branch with if/else and remember state.

An if statement runs a block when its condition is true. An optional else block runs when the condition is false. Equality uses ==; assignment uses =.

```
if (buttonState == LOW) { tone(3, 800); }
else { noTone(3); }
```

Both branches should establish a complete, safe output state.

Trace the source example twice

In the given code, LOW makes the equality comparison true, so tone(3, 800) runs. A different value selects else and noTone(3). The 800 belongs to the source code example; current classroom sound-testing directions still control practical use. Notice the two equals signs. Replacing == with = changes a comparison into assignment, so a visually small edit can change the program’s meaning.

17. Which correction best addresses the misconception “if and else run one after the other every time”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C only the branch selected by the condition runs on that pass

State the selected command for buttonState HIGH in the given example.

Continue online: Learning activity 6.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

6.3 A VARIABLE CAN REMEMBER STATE

A variable can remember state

Read the button, branch with if/else and remember state.

A toggle changes an output once per button press: off to on, then on to off. A Boolean variable can remember the current state using true or false.

One physical press can produce several rapid readings, called switch bounce. The supplied activity may use a simple release check or timing strategy; implement only the teacher-directed method and explain its purpose.

A reading is not a press event

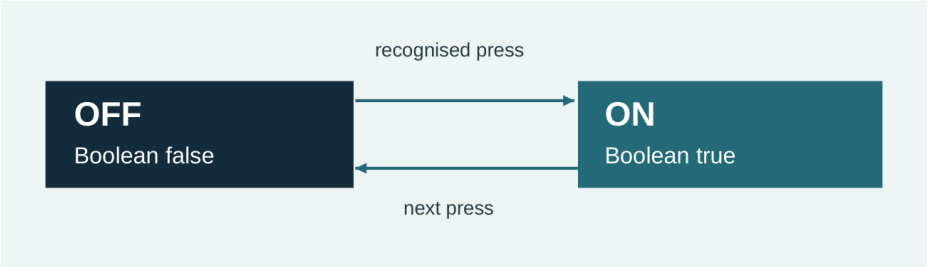


Figure 6.3. Two recognised presses invert the same stored Boolean twice.

The source toggle model starts with false, becomes true after one recognised press, then returns to false after the next. The stored Boolean keeps the state between loop cycles. Repeated readings during one physical press must not be mistaken for separate intended presses. Record the teacher-directed recognition or bounce-control method and explain how it helps one action produce one state change.

18. One press appears to switch the LED several times. What should happen next?

- ○ A check how the program recognises a single press and applies the approved bounce control
- ○ B set the state true on every loop while the button remains held
- ○ C hide the result and remove it from the record

Explain the state changes in a one-button LED toggle.

Continue online: Learning activity 6.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10.

Open the matching course section

SOURCE PATHWAY 8–9: PUSH BUTTON BASICS AND GET A TOGGLE ON.

Button and toggle laboratory 1 of 2

Observe the real pin 7 states, build a complete if/else rule and trace stored toggle state.

1. Read the truth table, then observe pin 7 when released and pressed instead of guessing.
2. Write the condition and define every output in both branches.
3. Use the state diagram to trace two recognised presses.
4. Record the teacher-directed press-recognition or bounce-control method separately from the core logic.

Value read from pin 7 when released.

Value read from pin 7 when pressed.

Write the correct pressed-state condition.

List every output in the true branch.

List every output in the false branch.

Open the matching course section

SOURCE PATHWAY 8–9: PUSH BUTTON BASICS AND GET A TOGGLE ON.

Button and toggle laboratory 2 of 2

Observe the real pin 7 states, build a complete if/else rule and trace stored toggle state.

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Trace the Boolean through two recognised presses.

Stage	Stored Boolean	Output
Before a press		
After first recognised press		
After second recognised press		

Record the teacher-directed press-recognition method.

Online extension: Build a truth table (busy-work.html#busy-11); Trace a toggle (busy-work.html#busy-12).

Open the matching course section

7.1 TONE CONTROLS FREQUENCY

Tone controls frequency

Control the buzzer with tone, mapping and musical sequences.

The buzzer on pin 3 can produce a tone using a chosen frequency. Frequency is measured in hertz and relates to perceived pitch: a higher frequency usually sounds higher.

tone(3, frequency) starts the sound; noTone(3) stops it. Classroom volume, duration and testing routines remain Teacher to confirm.

A tone needs a stopping point

```
tone(3, frequency);
noTone(3);
```

Source frequency	Relative pitch in the example
150 Hz	Low
550 Hz	Mid-range
1200 Hz	High

The source frequency scale illustrates relative pitch, not permission to test these values. noTone(3) stops sound; the teacher confirms the actual sound-testing controls.

A numeric frequency controls the signal sent to the buzzer on pin 3. Changing that value changes pitch; it is not a substitute for a classroom volume or duration rule. Find both the start command and noTone(3) in the program so silence is part of the intended sequence. Keep practical tests short or otherwise controlled exactly as directed by the teacher, without inventing a duration.

19. Which action best applies the theory?

- ○ A leave an unnecessary continuous tone running during all testing
- ○ B change several unrelated parts and hope
- ○ C send a suitable frequency to buzzer pin 3 and stop it deliberately

Explain how the program starts, changes and stops a buzzer tone.

Continue online: Learning activity 7.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3.

Open the matching course section

7.2 SENSOR DATA CAN CONTROL PITCH

Sensor data can control pitch

Control the buzzer with tone, mapping and musical sequences.

A pitch changer reads the potentiometer, maps its input range to a useful frequency range and sends that result to the buzzer. The signal path is A5 → analogRead → map → tone on pin 3.

Choose the output range by testing, not by assuming that every numeric frequency is equally useful for the classroom task.

Trace the number through the system

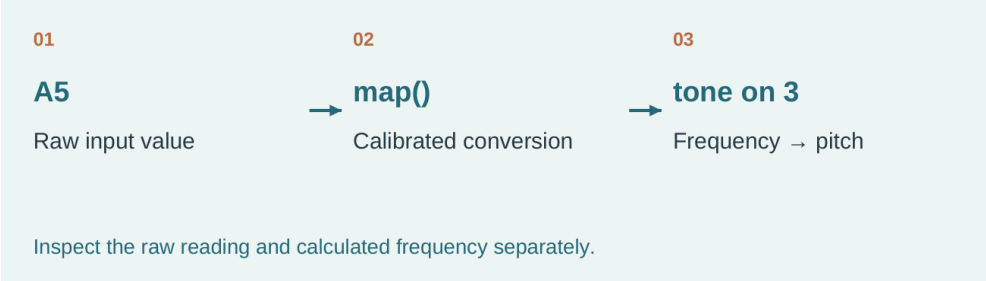


Figure 7.2. The raw A5 value and mapped frequency have different meanings along one signal path.

Turning the potentiometer changes an A5 reading. The map calculation converts the calibrated input range into the chosen frequency range. tone uses the resulting number on pin 3. If the pitch does not change as expected, inspect the reading, the mapped value and the output separately. This keeps a mapping error distinct from a sensor-reading or buzzer-command error.

20. Which correction best addresses the misconception “map creates sound by itself”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C map calculates a number and tone uses that number as frequency

Name the value you would inspect between reading A5 and calling tone.

Continue online: Learning activity 7.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

7.3 SEQUENCES CAN CREATE A MELODY

Sequences can create a melody

Control the buzzer with tone, mapping and musical sequences.

A melody is an ordered sequence of note frequencies and durations. Arrays can store related notes; a loop can visit each position; a function can keep repeated playing steps together.

Trace completed source code before changing it. The learning purpose is to identify iteration, arrays and a function—not to copy an unexplained song.

Separate the three jobs

Part	Job
Array	Stores note values and related data
Loop	Visits each position in order
Function	Groups the playing steps

First note	Second note	Third note
LOW pitch • 300 ms	HIGH pitch • 300 ms	LOW pitch • 300 ms

Source low–high–low example: start tone, play the three stated durations, stop tone, then repeat the pattern. LOW/HIGH here describe relative pitch, not digitalWrite logic states.

An array is the stored sequence of note values. The loop chooses each position in order. A function carries out the playing steps in one named place, using frequency and duration. These jobs cooperate but are not interchangeable: visiting the next index does not explain how a note is played. Trace the supplied class code before changing it so a copied melody becomes an explained sequence.

21. The notes array has eight items but the loop attempts nine positions. What should happen next?

- ○ A correct the loop boundary so it visits only valid indexes
- ○ B change one array length without checking the other
- ○ C hide the result and remove it from the record

Which part organises order, and which part packages the playing steps?

Continue online: Learning activity 7.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

SOURCE PATHWAY 10–12: BUZZER BASICS, PITCH CHANGER AND PLAY A SONG.

Buzzer and pitch laboratory

Connect frequency, mapped sensor data and note duration to a controlled audible signal.

1. Read the frequency scale on page 31 and the low–high–low sequence on page 33 before testing sound.
2. Follow current teacher directions for short buzzer tests and record one frequency with its perceived pitch.
3. Trace the signal from A5 through mapping to tone on pin 3.
4. Explain where noTone stops sound and how the array, loop and function create an ordered sequence.

Record one tested frequency and perceived pitch.

Where does the program call noTone?

State the calibrated A5 range.

State the tested mapped frequency range.

Trace A5 data to buzzer pitch.

Explain the jobs of the note array, loop and function.

Current classroom sound-testing directions: Teacher to confirm.

Online extension: Tune the buzzer (busy-work.html#busy-13); Compose a coded signal (busy-work.html#busy-14).

Open the matching course section

8.1 ARRAYS ORGANISE RELATED VALUES

Arrays organise related values

Use arrays, functions and iteration in multi-output challenges.

An array stores several values under one name, with each position identified by an index. The first item uses index 0. An LED pin array can hold the six fixed output pins 8–13.

Arrays are useful when the same operation must be applied to a set of related components.

Six values have six positions

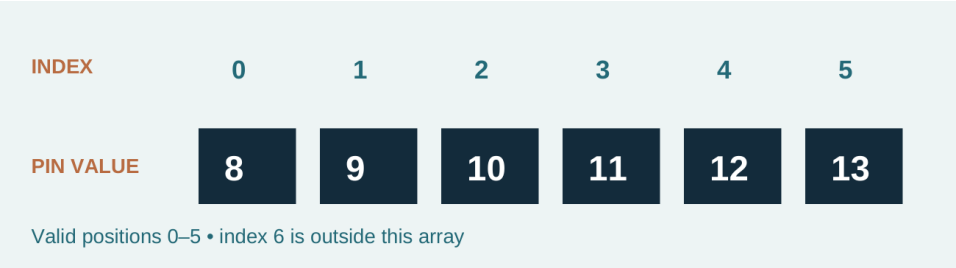


Figure 8.1. The six fixed LED pins are values; positions 0–5 are indexes.

With the pins stored in the order 8, 9, 10, 11, 12, 13, index 0 selects pin 8 and index 5 selects pin 13. Index 6 would be outside that six-item array. The source activity also shows a shorter four-item example, pins 8–11 at indexes 0–3. That teaches the same indexing rule; it is not a replacement for the full fixed LED map.

22. Which action best applies the theory?

- ○ A treat the sixth item as index 6 in a six-item array
- ○ B change several unrelated parts and hope
- ○ C define the six pins and access only valid indexes from 0 to 5

Identify the pin at index 4 in the six-item array shown.

Continue online: Learning activity 8.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

8.2 LOOPS AND FUNCTIONS REDUCE REPETITION

Loops and functions reduce repetition

Use arrays, functions and iteration in multi-output challenges.

A for loop repeats a block while updating a counter. A function groups a named task that can be called when needed. Together they can switch several LEDs without duplicating six nearly identical commands.

Reducing repetition is useful only when the result remains readable and the loop boundaries are correct.

Stop before the first invalid index

Visit	Array index	Selected pin
First	0	8
Last valid	5	13
Outside the six-item array	6	No valid item

A loop using a six-item LED array must visit indexes 0, 1, 2, 3, 4 and 5. The final valid visit is not index 6. Trace the counter, selected pin and operation together so the boundary can be checked. A function can name the repeated task, but the name alone does not prove the loop is correct. Read the first and final iterations before running the combined system.

23. Which correction best addresses the misconception “a loop always makes code easier regardless of its design”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C the loop must be bounded, readable and suited to repeated work

Explain why visiting index 6 is different from controlling pin 6.

Continue online: Learning activity 8.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

8.3 COMBINED CHALLENGES EXPOSE INTERACTIONS

Combined challenges expose interactions

Use arrays, functions and iteration in multi-output challenges.

Electronic Dice, Buzzing Light Meter and LED Magic Sign combine earlier ideas. A useful system description separates its input, processing rules and outputs before examining individual code lines.

When a combined build fails, divide it into subsystems. Test the input reading, the calculation or branch, then each output. Changing many parts at once hides evidence.

Split a fault into smaller questions

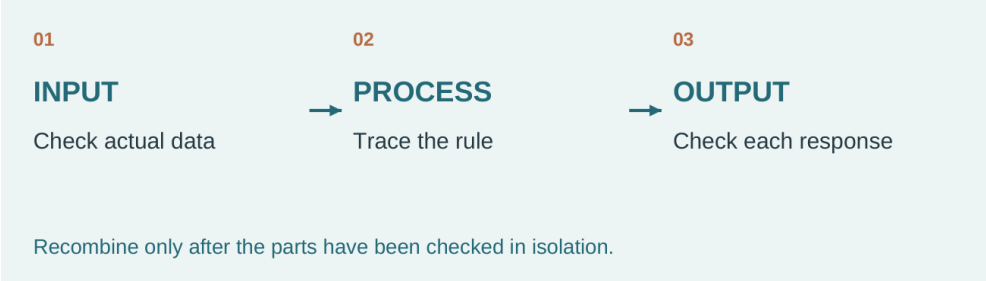


Figure 8.3. Test input, processing and outputs separately before recombining them.

If a Buzzing Light Meter gives an unexpected response, first ask whether its input reading is sensible. Then inspect the mapping or branch; finally inspect each output. A fault seen only after combining parts may be an interaction rather than a broken component. Record the test order and change one cause at a time. Use the actual Electronic Dice, Buzzing Light Meter or LED Magic Sign completed in class.

24. The light meter reads sensible A4 data but no LEDs illuminate. What should happen next?

- ○ A keep the input evidence and test mapping, level calculation and LED outputs in order
- ○ B rewrite the whole sketch after one unexplained symptom
- ○ C hide the result and remove it from the record

Why test a subsystem before changing several parts of the combined program?

Continue online: Learning activity 8.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

SOURCE PATHWAY 13–15: ELECTRONIC DICE, BUZZING LIGHT METER AND LED MAGIC SIGN.

Combined-system challenge lab 1 of 2

Decompose a combined challenge, trace its array correctly and test one subsystem at a time.

1. Use the array diagram to confirm valid indexes, then use the subsystem diagram to plan the test order.
2. Choose the combined challenge actually completed in class.
3. Identify its input, processing rules, outputs and any array used.
4. Test input, processing and outputs separately before recombining; record one interaction fault and correction.

Choose the combined challenge completed.

Identify its input subsystem.

Identify its processing rules.

Identify every output.

Explain one array and its valid indexes.

Record the actual values and valid indexes for your chosen challenge.

Open the matching course section

SOURCE PATHWAY 13–15: ELECTRONIC DICE, BUZZING LIGHT METER AND LED MAGIC SIGN.

Combined-system challenge lab 2 of 2

Decompose a combined challenge, trace its array correctly and test one subsystem at a time.

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Trace a for loop that visits all six LED pins in an array.

Use the matching studio fields below for this complete guided response. One record serves both purposes.

Trace the first and final loop iteration.

Visit	Index / value	Action
First		
Second		
Third		
Fourth		
Fifth		
Final		

Write the subsystem test order.

Record one interaction fault and correction.

Online extension: Trace an array loop (busy-work.html#busy-15); Combine and test subsystems (busy-work.html#busy-16).

Open the matching course section

9.1 TRANSLATE THE BRIEF INTO CRITERIA

Translate the brief into criteria

Turn the supplied brief into a planned, coded and tested alert system.

The final challenge is to design, produce and evaluate a ThinkerShield alarm or alert system that protects something you care about. It must use the fixed available inputs and outputs, with outputs changing according to input data.

Write testable criteria for function, clarity, reliability and explanation. Current marks, due date, rubric and submission route remain Teacher to confirm.

Make success observable

Vague claim	Observable question
It is reliable	Does it repeat the intended response in the defined tests?
It resets	Do normal outputs return after the trigger is removed?

“A good alarm” is too broad to test. A criterion must name a behaviour or quality and how evidence will establish it. For example, a recovery criterion asks whether the intended normal outputs return after the trigger condition is removed. It does not prescribe an unverified sensor threshold or formal mark. Write at least five criteria that relate to the actual thing protected and the people using the system.

25. Which action best applies the theory?

- ○ A use vague claims such as “the alarm is good”
- ○ B change several unrelated parts and hope
- ○ C write observable criteria linked to the user, purpose and required system behaviour

Turn “it is reliable” into a criterion that could be checked.

Continue online: Learning activity 9.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

9.2 PLAN THE COMPLETE SIGNAL PATH

Plan the complete signal path

Turn the supplied brief into a planned, coded and tested alert system.

Before coding, decide the job of each input and output. Build an IPO diagram, pseudocode and flowchart that agree with one another and the fixed pin map.

Plan normal, warning and alert states where useful. Each state should explicitly set all relevant LEDs and the buzzer so old outputs do not remain active accidentally.

Check one signal in every representation

PIN MAP	Which address does each part use?
IPO	What data, decision and response connect?
PSEUDOCODE	Are both paths and repetition complete?
CODE	Does the tested version implement that plan?

Figure 9.2. Four representations should describe the same complete system.

Choose one input and follow it from the pin map through the IPO description, the pseudocode and the code. Each representation should name the same rule and resulting outputs. A mismatch is useful evidence: the plan may call for recovery while the code never clears the buzzer. Resolve the disagreement before relying on a polished flowchart or a successful single demonstration.

26. Which correction best addresses the misconception “planning ends once code is typed”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C the plan remains a reference for testing and justified revision

What could a code trace reveal that an attractive IPO drawing does not prove?

Continue online: Learning activity 9.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

9.3 TEST AGAINST PREDICTIONS

Test against predictions

Turn the supplied brief into a planned, coded and tested alert system.

A useful test log states the input condition, expected output, actual output and next action. Include normal, warning, alert and recovery conditions plus values near any threshold.

Change one factor at a time and keep evidence. A failed test is useful when it reveals a specific difference between prediction and result.

Write the prediction before the result

Test record	Meaning
Input condition	What was supplied or observed
Expected output	Prediction before testing
Actual output	Result observed during testing
Next action	What the difference means

In a test log, expected output states what should happen; actual output states what was observed. Keeping them separate prevents a failed result from being quietly re-labelled a success. Include normal and trigger conditions, a boundary case and recovery. Where warning is part of the design, test it too. A useful next action names the discrepancy or confirms the criterion that the evidence supports.

27. The alarm triggers correctly but never returns to normal. What should happen next?

- ○ A record the failure, inspect the recovery branch, change one cause and retest
- ○ B write “tested” without conditions or observations
- ○ C hide the result and remove it from the record

Why should actual output not be copied from the expected-output column?

Continue online: Learning activity 9.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

SOURCE PATHWAY 16: FINAL ALARM CODE.

Final alarm design studio 1 of 3

Turn the final alarm need into aligned criteria, representations, code evidence and test cases.

1. Use the need, criterion and test explanation on page 40 to write a focused brief and at least five observable criteria.
2. Use the alignment diagram to cross-check the pin map, IPO description, pseudocode and code.
3. Plan normal, trigger, boundary and recovery tests before judging the design.
4. Record code location and feedback only through the current teacher-confirmed process.

What will the alarm protect and for whom?

Write at least five testable criteria.

Criterion	What observable success will look like
1	
2	
3	
4	
5	

Write the final pin map.

Input or output	Fixed pin(s) / role
Button	
LDR	
Potentiometer	
LEDs	
Buzzer	

Open the matching course section

FINAL ALARM CODE • EVIDENCE 2 OF 3

Make the representations agree

Continue the same studio from page 43; keep one connected record.

Use your brief, criteria and pin map on page 43. The IPO description, normal/alert pseudocode and code must describe the same system. Set every relevant output in each state so an old alert does not remain active accidentally.

Describe the complete input–process–output flow.

Label each input, the rule and each output. Use arrows to show the information path.

Write complete normal and alert pseudocode.

Show reading, the decision, both paths and repetition. This is planning space, not an assumed teacher-approved algorithm.

Paste the important code or record the teacher-confirmed file location.

Use the location option when the code needs more space. Check that its version matches this plan. Five test cases and authorised feedback continue on page 45.

Fixed map: button 7 • LDR A4 • potentiometer A5 • LEDs 8–13 • buzzer 3.

Open the matching course section

SOURCE PATHWAY 16: FINAL ALARM CODE.

Final alarm design studio 3 of 3

Turn the final alarm need into aligned criteria, representations, code evidence and test cases.

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Create a five-case alarm test plan including boundary and recovery evidence.

Use the matching studio fields below for this complete guided response. One record serves both purposes.

Write five test cases including boundary and recovery.

Condition	Expected	Actual	Next action

Record teacher or peer feedback if authorised.

Online extension: Translate the brief (busy-work.html#busy-17); Align every representation (busy-work.html#busy-18).

Open the matching course section

10.1 DEBUG ONE CAUSE AT A TIME

Debug one cause at a time

Use evidence to correct faults and evaluate the finished system.

Debugging is a systematic process: reproduce the fault, compare the observed and expected result, isolate the likely stage, change one cause and retest.

Compiler messages often point near a syntax problem, not always exactly at it. Check the reported line, the line before it, matching braces, semicolons, names and pin references.

Retest the same situation

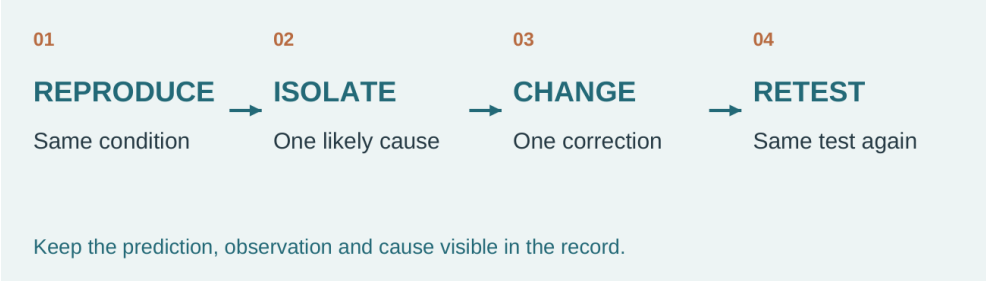


Figure 10.1. Return to the same test after changing one likely cause.

A correction is convincing when the original fault can be reproduced, one likely cause is changed, and the same test is repeated. If the test condition changes as well, success may have another explanation. Keep compiler messages and the surrounding code in view: the line named by a message may be near the cause. Check the previous line, braces, semicolons and names before making unrelated edits.

28. Which action best applies the theory?

- ○ A change several unrelated lines at once
- ○ B change several unrelated parts and hope
- ○ C reproduce, isolate, change one cause and retest

Why does one successful result after several unrelated edits give weak debugging evidence?

Continue online: Learning activity 10.1, questions 1, 2, 4, 5, 6, 7, 8, 9, 10. Printed check = website question 3. The complete additional written response is online.

Open the matching course section

10.2 EVALUATION USES CRITERIA AND EVIDENCE

Evaluation uses criteria and evidence

Use evidence to correct faults and evaluate the finished system.

Evaluation is a judgement of how well the solution met the brief and criteria. Use observations, test results, code traces and user feedback where authorised.

For each important criterion, state the evidence, make a judgement and identify a specific improvement. “It worked” is incomplete because it does not show which conditions were tested.

Connect the judgement to a test

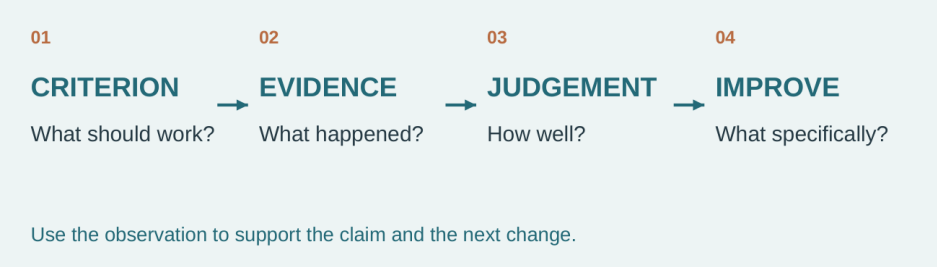


Figure 10.2. A defensible judgement links a criterion to observed evidence and an improvement.

“It worked” gives no test condition and no criterion. A stronger evaluation names the criterion, cites a relevant observation, judges how well it was met and identifies a precise improvement. If recovery failed, say so and explain what should change. Honest unsuccessful evidence is more useful than an unsupported success claim because it gives the next design decision a reason.

29. Which correction best addresses the misconception “evaluation means saying whether you enjoyed the task”?

- ○ A the misconception is always correct
- ○ B evidence and definitions are unnecessary
- ○ C personal reflection can contribute, but solution quality must be judged against evidence

What two pieces of information are missing from “it worked”?

Continue online: Learning activity 10.2, questions 1, 2, 3, 4, 5, 6, 7, 8, 10. Printed check = website question 9. The complete additional written response is online.

Open the matching course section

10.3 RESPONSIBLE DIGITAL PRACTICE PROTECTS WORK

Responsible digital practice protects work

Use evidence to correct faults and evaluate the finished system.

Save named versions of code as directed and keep a backup before major changes. This website stores practice responses only in the current browser; it does not upload to Drive or submit to Classroom.

Explain borrowed ideas and keep your final code understandable. Current file naming, Drive location, Classroom turn-in and collaboration rules remain Teacher to confirm.

Saved here is not submitted

Record	What it establishes
Browser response	Practice saved on that browser/device
Code version and backup	A named recoverable copy of the tested work
Formal submission	Only the current teacher-confirmed process

This website saves practice responses in the current browser. That is not the same as uploading a code version, keeping a separate backup or formally turning in an assignment. Check that the code version in your record matches the code tested. Follow the teacher's current naming, location and collaboration directions, explain borrowed ideas and keep the code understandable enough to discuss.

30. A student is changing device after completing written responses. What should happen next?

- ○ A download the folio backup and follow the teacher-confirmed transfer and submission method
- ○ B overwrite the only working copy before an untested change
- ○ C hide the result and remove it from the record

Explain one difference between a backup and a formal hand-in.

Continue online: Learning activity 10.3, questions 1, 2, 3, 4, 5, 6, 7, 8, 9. Printed check = website question 10. The complete additional written response is online.

Open the matching course section

USE PROJECT EVIDENCE TO DEBUG, EVALUATE AND BACK UP.

Testing and evaluation builder 1 of 2

Use one-change debugging and criterion evidence to evaluate the finished alarm honestly.

1. Follow the debugging cycle from observed fault to one likely cause, one change and a repeat of the same test.
2. Use the evaluation diagram to connect each criterion to actual test evidence and a defensible judgement.
3. Recommend precise improvements that respond to the recorded evidence.
4. Record the code version and folio backup; the formal submission route remains Teacher to confirm.

Describe one reproducible fault.

State the likely cause.

Record the one change made.

Record the retest result.

Open the matching course section

USE PROJECT EVIDENCE TO DEBUG, EVALUATE AND BACK UP.

Testing and evaluation builder 2 of 2

Use one-change debugging and criterion evidence to evaluate the finished alarm honestly.

Continue the same studio. Use the earlier plan and record actual observations; these pages are one connected task.

Evaluate the final alarm against at least five criteria using test evidence.

Use the matching studio fields below for this complete guided response. One record serves both purposes.

Evaluate the alarm against at least five criteria.

Criterion ref.	Test evidence	Judgement

Name three precise improvements.

Explain the most useful coding idea learned.

Record the code version and folio backup completed.

Current submission route: Teacher to confirm.

Online extension: Debug from evidence (busy-work.html#busy-19); Evaluate and improve (busy-work.html#busy-20).

Open the matching course section